1. Identifying Unique Characters in a String

```
public class UniqueCharacters {
    public static void main(String[] args) {
        String input = "programming";
        for (char c : input.toCharArray()) {
            if (input.indexOf(c) == input.lastIndexOf(c)) {
                System.out.print(c + " ");
            }
        }
    }
}
```

Explanation: Prints characters that appear only once by comparing first and last index.

2. Identifying Duplicate Characters in a String

```
import java.util.HashMap;

public class DuplicateCharacters {
    public static void main(String[] args) {
        String input = "programming";
        HashMap<Character, Integer> map = new HashMap<>();
        for (char c : input.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        for (char c : map.keySet()) {
            if (map.get(c) > 1) {
                System.out.print(c + " ");
            }
        }
    }
}
```

Explanation: Uses HashMap to count character frequencies and prints duplicates.

3. Reverse a String

```
public class ReverseString {
    public static void main(String[] args) {
        String input = "hello";
        String reversed = new StringBuilder(input).reverse().toString();
        System.out.println("Reversed: " + reversed);
    }
}
```

Explanation: Uses StringBuilder's reverse() method to reverse the string.

4. Check if a String is Palindrome

```
public class PalindromeCheck {
    public static void main(String[] args) {
        String input = "madam";
        String reversed = new StringBuilder(input).reverse().toString();
        if (input.equals(reversed)) {
```

```
            System.out.println("Palindrome");
        } else {
            System.out.println("Not a palindrome");
        }
    }
}
```

Explanation: Compares original string with its reversed version.

5. Count Vowels and Consonants

```
public class VowelConsonantCount {
    public static void main(String[] args) {
        String input = "hello world";
        int vowels = 0, consonants = 0;
        for (char c : input.toLowerCase().toCharArray()) {
            if (Character.isLetter(c)) {
                if ("aeiou".indexOf(c) != -1) vowels++;
                else consonants++;
            }
        }
        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
    }
}
```

Explanation: Counts vowels and consonants using character checks.

6. Find First Non-Repeated Character

```
import java.util.LinkedHashMap;

public class FirstNonRepeated {
    public static void main(String[] args) {
        String input = "swiss";
        LinkedHashMap<Character, Integer> map = new LinkedHashMap<>();
        for (char c : input.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        for (char c : map.keySet()) {
            if (map.get(c) == 1) {
                System.out.println("First non-repeated: " + c);
                break;
            }
        }
    }
}
```

Explanation: Uses LinkedHashMap to maintain order and find first unique character.

7. Remove Whitespace from a String

```
public class RemoveWhitespace {
    public static void main(String[] args) {
        String input = " hello world ";
```

```
        String result = input.replaceAll("\s+", "");
        System.out.println("Without spaces: " + result);
    }
}
```

Explanation: Uses regex to remove all whitespace characters.

8. Swap Two Strings Without Using Third Variable

```
public class SwapStrings {
    public static void main(String[] args) {
        String a = "hello";
        String b = "world";
        a = a + b;
        b = a.substring(0, a.length() - b.length());
        a = a.substring(b.length());
        System.out.println("a: " + a);
        System.out.println("b: " + b);
    }
}
```

Explanation: Swaps two strings using concatenation and substring.

9. Check if Two Strings are Anagrams

```
import java.util.Arrays;

public class AnagramCheck {
    public static void main(String[] args) {
        String s1 = "listen";
        String s2 = "silent";
        char[] a1 = s1.toCharArray();
        char[] a2 = s2.toCharArray();
        Arrays.sort(a1);
        Arrays.sort(a2);
        if (Arrays.equals(a1, a2)) {
            System.out.println("Anagrams");
        } else {
            System.out.println("Not anagrams");
        }
    }
}
```

Explanation: Sorts and compares character arrays to check for anagrams.

10. Find Longest Substring Without Repeating Characters

```
import java.util.HashSet;

public class LongestUniqueSubstring {
    public static void main(String[] args) {
        String s = "abcabcbb";
        int maxLen = 0;
        for (int i = 0; i < s.length(); i++) {
            HashSet<Character> set = new HashSet<>();
```

```
            int j = i;
            while (j < s.length() && !set.contains(s.charAt(j))) {
                set.add(s.charAt(j));
                j++;
            }
            maxLen = Math.max(maxLen, j - i);
        }
        System.out.println("Length: " + maxLen);
    }
}
```

Explanation: Uses sliding window with HashSet to find longest unique substring.